Information Processing Letters 33 (1989) 187–191
North-Holland

21 December 1989

A NOTE ON ADAPTIVE PARALLEL SORTING

Christos LEVCOPOULOS and Ola PETERSSON

Department of Computer Science, Lund University, S-221 00 Lund, Sweden

Communicated by R. Wilhelm
Received 9 May 1989
Revised 16 August 1989

We present a cost optimal parallel algorithm for sorting presorted sequences. The measure of presortedness we consider is *Rem*, i.e., the smallest number of elements whose removal leaves a sorted sequence. The algorithm sorts a sequence X of length n, with $Rem(X) = r$, in $O(\log n)$ time, provided $O((n + r \log r)/\log n)$ processors are available, in the EREW PRAM model. This is the first PRAM sorting algorithm which is cost optimal with respect to *Rem*.

Keywords: Sorting algorithm, PRAM, presortedness

1. Introduction

A *measure of presortedness* is an integer function on a permutation π of a totally ordered set that reflects how much π differs from the total order. Examples of presortedness measures include the number of runs or inversions. The term presortedness was coined by Mehlhorn [17], who used the number of inversions in the sequence as a measure. Sequential algorithms for sorting presorted sequences have received a lot of attention [5,7,8,13,14,16,17–19]. Mehlhorn [17,18] gave an algorithm, A-Sort, which is adaptive with respect to the number of inversions. Intuitively, an algorithm is *adaptive* with respect to a measure of presortedness if it sorts all permutations, but performs particularly well on permutations that have a high degree of presortedness (without knowing the value of the measure beforehand). Mannila [16] formalized the concept of presortedness and gave algorithms that are adaptive with respect to several measures. Recently, Estivill-Castro and Wood [8], Skiena [19], and Levcopoulos and Petersson [13] have considered other measures. Cook and Kim [5] and Mannila [15] have studied the problem of designing adaptive sorting algorithms empirically.

The measure considered in this paper is *Rem*, i.e., the smallest number of elements whose removal leaves a sorted sequence. Mannila [16] has shown that $\Omega(n + r \log r)$ is a lower bound for sequential comparison-based algorithms that sort a sequence X of length n, with $Rem(X) = r$. The paper of Mannila also presents a sorting algorithm, Local Insertion Sort, whose upper bound matches this lower bound, and thus is optimal with respect to *Rem*. The paper of Cook and Kim [5] uses *Rem* as a measure of presortedness as well. Algorithms for calculating *Rem* have been developed by Dijkstra [6] and Fredman [9].

Optimal parallel sorting algorithms, which do not utilize presortedness, have been known since 1983, when Ajtai et al. [1] gave an $O(n \log n)$ sorting network, which can be simulated by an EREW PRAM (Exclusive Read Exclusive Write Parallel Random Access Machine). The running time of the algorithm was $O(\log n)$; however, the implicit constant factor was huge. Cole [4] gave a parallel implementation of Merge Sort [18], which sorts n elements in time $O(\log n)$, by using a linear number of processors, in the EREW PRAM model. Cole's algorithm, which we will refer to as Parallel Merge Sort, did not have such a large constant.

0020-0190/89/$3.50 © 1989, Elsevier Science Publishers B.V. (North-Holland)

Parallel sorting algorithms for presorted sequences, in contrast, have not been so deeply studied. To our knowledge, the only result is an EREW PRAM algorithm which is adaptive to the number of inversions [12]. In this paper, we provide the first cost optimal parallel sorting algorithm, with respect to *Rem*, in any PRAM model. The model used is the weakest among the different PRAMS, i.e., the EREW PRAM. The algorithm sorts a sequence X of length n, with $Rem(X) = r$, in time O(log n) by using O($(n + r \log r)/\log n$) processors on an EREW PRAM, which is cost optimal. The optimality stems from the fact that a P-processor, T-time, parallel algorithm can be implemented to run in time O(PT) on a single processor.

The algorithm first divides the input sequence into subsequences of length log n. In each subsequence, pairs of unordered elements are removed and the subsequences are permuted such that their medians are sorted. Now, elements that make the catenation of consecutive subsequences nonsorted are removed. All removed elements are then sorted in parallel and finally merged with the catenated subsequences. The algorithm uses both Parallel Merge Sort [4] and parallel prefix computation [10,11] as subroutines.

2. The algorithm

We start this section by giving some preliminary definitions. Let $X = \langle x_1, \ldots, x_n \rangle$ be a *sequence* of elements x_i from some linear order, that is, for all i, $j \in \{1, \ldots, n\}$, $x_i \leqslant x_j$ or $x_j \leqslant x_i$. If $X = \langle x_1, \ldots, x_n \rangle$ and $Y = \langle y_1, \ldots, y_m \rangle$ are two sequences, their *catenation*, denoted XY, is defined as $\langle x_1, \ldots, x_n, y_1, \ldots, y_m \rangle$. The *length* of X is denoted $|X|$. If $Z = \langle x_{f(1)}, x_{f(2)}, \ldots, x_{f(m)} \rangle$, $m \leqslant n$, and $f\{1, \ldots, m\} \to \{1, \ldots, n\}$ is injective and monotonically increasing, then Z is called a *subsequence* of X. In particular, Z is a *consecutive subsequence* of X if there exists an i, $1 \leqslant i \leqslant n - m + 1$, such that $Z = \langle x_i, \ldots, x_{i+m-1} \rangle$.

Let $X = \langle x_1, \ldots, x_n \rangle$ be the sequence to be sorted. The algorithm consists of two phases: first, elements out of order are removed from X; second, the removed elements are sorted in parallel and merged with the nonremoved elements.

Phase A: Allocate $P = n/\log n$ processors.

1. X is divided into consecutive subsequences X_i, $1 \leqslant i \leqslant \lceil n/\log n \rceil$, of length log n. Each processor scans its subsequence of log n elements sequentially and removes pairs of unordered elements, such that the remaining elements form a sorted subsequence of X_i. This can be done as follows. We scan the subsequence forward while keeping track of an *upsequence* (ascending subsequence). The upsequence, which initially contains the first element in X_i, is updated by comparing its head (last element) with the next element encountered during the scan, x_j. If x_j is smaller, both x_j and the head are removed, and the former head becomes the new head. Otherwise, no element is removed and x_j is made the new head. It is clear that, when finished, what remains in X_i is an ascending subsequence of X_i. If more than half of the elements in a subsequence are removed, the whole subsequence is removed.
2. Sort the medians of the remaining subsequences.
3. Permute these subsequences such that the medians are sorted. Let us denote the new subsequences by X_1', X_2', ...
4. For each pair of subsequences X_i' and X_{i+1}', if $|X_i'| > |X_{i+1}'|$, remove elements from the end of X_i' such that the number of elements in X_i' which are greater than its median equals the number of elements in X_{i+1}' which are less than *its* median. The case when $|X_i'| < |X_{i+1}'|$ is handled analogously. Next, remove the same number of elements from the end of X_i' as from the front of X_{i+1}', such that
   (i) $X_i'X_{i+1}'$ is sorted, and
   (ii) if fewer elements are removed $X_i'X_{i+1}'$ is not sorted.

End of Phase A.

Let m be the number of elements which were removed during Phase A.

Phase B: Allocate $P = (n + m \log m)/\log n$ processors.

1. Sort the removed elements.

2. Merge the sorted elements left in X with the removed elements.

End of Phase B.

2.1. Analysis and details

We start by showing that each step in Phase A takes time no more than O(log n) if $P = n/\log n$ processors are available. It is assumed that the sequence is stored, in order, in an array. (Otherwise, if the elements are stored in a linked list, we can start by performing list ranking [1], in time O(log n) by using O($n/\log n$) processors [2], which gives us enough information to transform the linked list into an array.)

(1) It is easily seen that all comparisons performed to determine which elements to remove can be done in total time O(log n). The actual removal of the elements and compaction of X can be done by applying a prefix sums computation [2] [10,11].

(2) Since the number of medians is not greater than the number of processors they can be sorted in time O(log n), by Parallel Merge Sort.

(3) Rearranging the subsequences can trivially be done in time O(log n), since we have at most log n elements per processor, and each processor can determine the destination of its elements.

(4) Assign one processor to each pair of subsequences, X_i' and X_{i+1}'. The initial balancing of the end of X_i' and X_{i+1}' guarantees that we can remove the same number of elements from X_i' and X_{i+1}' to satisfy (i) and (ii). Each processor can then determine the elements which shall be removed from the end of X_i' and from the front of X_{i+1}' within the required time bound. The removal and compaction are performed as in Step 1. We conclude that O(log n) time suffices for this step.

In Phase B, we have $P = (n + m \log m)/\log n$ processors, where m is the number of removed elements. Step 1 is performed in O(log n) time by applying Parallel Merge Sort. Further, Bilardi and Nicolau [3] have shown that merging two sorted sequences of length at most n can be done in time O(log n), by using O($n/\log n$) processors in the EREW and PRAM model, which suffices for Step 2.

Since both phases run in time O(log n) we have

Lemma 1. *The given algorithm runs in time* O(log n), *if* $P = (n + m \log m)/\log n$ *EREW PRAM processors are available.*

To prove that the algorithm is optimal with respect to *Rem*, we need another lemma.

Lemma 2. $m \leqslant 8\ Rem(X)$.

Proof. Let p and q denote the number of removed elements in Step 1 and Step 4, respectively.

It is clear that, in Step 1, the number of elements belonging to unordered pairs, $p_1 \leqslant 2\ Rem(X)$, since at least one element in each pair must be out of order. Denote by p_2 the number of elements removed due to more than half-empty subsequences. Thus, $p \leqslant 2\ Rem(X) + p_2$.

In Step 4, the number of elements removed to make the lengths (above and below the median) equal, $q_1 \leqslant p_1 - p_2$. Let $q_2(i)$ denote the number of elements removed from the end of X_i' and from the front of X_{i+1}' when removing pairs in Step 4. We distinguish two cases:

(a) X_i' and X_{i+1}' are in the same order as before Step 3. In this case, at least one element from each pair removed must have been out of order originally. Hence, the total contribution by all such $q_2(i)$'s is bounded by $2\ Rem(X)$.

(b) X_i' and X_{i+1}' changed order in Step 3. Then, all elements less than the median in X_i' are smaller than all the elements greater than the median in X_{i+1}'. Therefore, at least half of these elements must have been out of order originally. Since each remaining subsequence contains at least $\log n/2$ elements, there are at least $\log n/4$ such elements which were out of order before Step 3. Further, $q_2(i) \leqslant \log n$. Thus, for at least every fourth element removed there is an element which was out of order originally. Therefore, the sum of

[1] The list ranking problem is defined as follows: Given a linked list of length n, for each element compute the number of elements following it in the linked list.

[2] The prefix sums problem is defined as follows: Given an array of n numbers $A(1), A(2), \ldots, A(n)$, compute $\Sigma_{j=1}^{i} A(j)$, for all $1 \leqslant i \leqslant n$.

all such $q_2(i)$'s is bounded by 4 $Rem(X)$.

Hence,

$$q = q_1 + \sum_i q_2(i) \leqslant p_1 - p_2 + 4\ Rem(X)$$
$$\leqslant 6\ Rem(X) - p_2$$

and we conclude,

$$m = p + q \leqslant 8\ Rem(X). \quad \square$$

Since $\Omega(n + Rem(X)\log(Rem(X)))$ is a sequential lower bound, and by Lemma 1 and Lemma 2, we have proved

Theorem 3. *Given a sequence X of length n, with Rem(X) = r, there is an EREW PRAM algorithm that sorts X in time* $O(\log n)$, *by using* $P = O((n + r \log r)/\log n)$ *processors, which is cost optimal.*

Remark 4. It should be observed that Step 3 actually can increase *Rem*, which can be seen by considering the following example. Let X_i' and X_{i+1}' be two subsequences of equal length which changed order in Step 3, and let i be approximately $n/(2 \log n)$. Assume that the elements smaller than the median of X_i' are the smallest in the whole sequence and that the elements greater than the median of X_{i+1}' are the greatest in the whole sequence. Then, all these elements will be out of order after as well as before Step 3. Moreover, assume that all elements in X_i' which are greater than its median are also greater than the median of X_{i+1}'. Furthermore, we assume that before Step 3 the only elements contributing to *Rem* are those coming from either X_i' or X_{i+1}'. That is,

$$Rem(X) = \tfrac{1}{2}|X_i'| + \tfrac{1}{2}|X_{i+1}'| + 1$$

before Step 3. Let X' denote the catenation of the permuted subsequences. Then

$$Rem(X') = \tfrac{1}{2}|X_i'| + \tfrac{1}{2}|X_{i+1}'| + \left(\tfrac{1}{2}|X_i'| - 1\right).$$

Hence, *Rem* might increase by about fifty percent, due to Step 3.

Remark 5. It should be observed that in Phase A of the algorithm, not all elements removed need actually be removed for the algorithm to work. In Step 1, we need not remove the more than half-empty subsequences, and in Step 4, it is enough to just remove the minimum number of elements so that the catenations of the subsequences form sorted sequences (this can be done in time $O(\log n)$). The reason for performing these extra removals is that it makes the proof of Lemma 2 simple An interesting question is, what the actual value of the constant in Lemma 2 is if the removal is performed as just outlined. We conjecture that the value is 2.

References

[1] M. Ajtai, J. Komlós, and E. Szemeredi, Sorting in $c \log n$ parallel steps, *Combinatorica* **3** (1983) 1–19.

[2] R.J. Anderson and G.L. Miller, Deterministic parallel list ranking, in: *Proc. 3rd Aegean Workshop on Computing*, Lecture Notes in Computer Science **319** (Springer, Berlin, 1988) 81–90.

[3] G. Bilardi and A. Nicolau, Adaptive bitonic sorting: An optimal parallel algorithm for shared memory models, *SIAM J. Comput.* **18** (2) (1989) 216–228.

[4] R. Cole, Parallel merge sort, *SIAM J. Comput.* **17** (4) (1988) 770–785.

[5] C.R. Cook and D.J. Kim, Best sorting algorithms for nearly sorted lists, *Comm. ACM* **23** (11) (1980) 620–624.

[6] E.W. Dijkstra, Some beautiful arguments using mathematical induction, *Acta Inform.* **13** (1980) 1–8.

[7] E.W. Dijkstra, Smoothsort, an alternative to sorting in situ, *Sci. Comput. Programm.* **1** (1982) 223–233.

[8] V. Estivill-Castro and D. Wood, A new measure of presortedness, Research Report CS-87-58, Univ. of Waterloo, Dept. of Computer Science, Waterloo, Canada, 1987.

[9] M.L. Fredman, On computing the length of longest increasing subsequences, *Discrete Math.* **11** (1975) 29–35.

[10] C.P. Kruskal, L. Rudolph and M. Snir, The power of parallel prefix, in: *Proc. 1985 International Conference on Parallel Processing* (1985) 180–185.

[11] R.E. Ladner and M.J. Fisher, Parallel prefix computation, *J. ACM* **27** (4) (1980) 831–838.

[12] C. Levcopoulos and O. Petersson, An optimal parallel algorithm for sorting presorted files, in: *Proc. 8th Conference on Foundations of Software Technology and Theoretical Computer Science*, Lecture Notes in Computer Science **338** (Springer, Berlin, 1988) 154–160.

[13] C. Levcopoulos and O. Petersson, Heapsort – adapted for presorted files, in: *Proc. 1989 Workshop on Algorithms and Data Structures*, Lecture Notes in Computer Science (Springer, Berlin, 1989).

[14] C. Levcopoulos and O. Petersson, Splitsort – an adaptive sorting algorithm, Technical Report, Dept. of Computer Science, Lund Univ., Lund, Sweden, 1989.

[15] H. Mannila, Implementation of a sorting algorithm suita-

ble for presorted files, Technical Report, Dept. of Computer Science, Univ. of Helsinki, Finland, 1984.

[16] H. Mannila, Measures of presortedness and optimal sorting algorithms, *IEEE Trans. Comput.* **34** (4) (1985) 318–325.

[17] K. Mehlhorn, Sorting presorted files, in: *Proc. 4th GI Conference on Theoretical Computer Science*, Berlin, Lecture Notes in Computer Science **67** (Springer, Berlin, 1979) 199–212.

[18] K. Mehlhorn, *Data Structures and Algorithms, Vol 1: Sorting and Searching* (Springer, Berlin, 1984).

[19] S.S. Skiena, Encroaching lists as a measure of presortedness, *BIT* **28** (1988) 775–784.